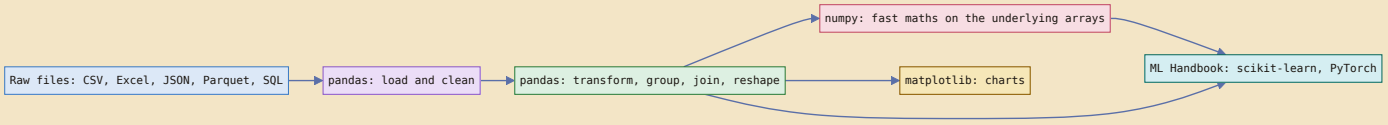
Chapter 18 — The Data Stack: numpy, pandas, matplotlib

What you will learn. The three libraries almost every Python data, science and ML (machine learning — programs that learn patterns from data instead of following hand-written rules) project stands on. **numpy**: fast n-dimensional arrays (grids of numbers with any number of dimensions), dtypes, indexing and views, broadcasting, vectorisation, aggregation along axes, basic linear algebra (the maths of vectors and matrices) and the modern random API. **pandas**: `Series` and `DataFrame`, reading and writing files, `loc` / `iloc`, filtering, `groupby` / `agg` / `transform`, `merge` / `concat`, `pivot_table` / `melt`, missing data, datetimes, why `apply` is slow, and performance tips — including what changed in **pandas 3.0** (string dtype, Copy-on-Write). **matplotlib**: Figure and Axes, the four workhorse charts, subplots, labels and saving images. The chapter ends with a small end-to-end analysis: messy CSV (comma-separated values — a plain-text table, one row per line) → clean → analyse → chart.

Where this leads. The ML Handbook, Chapter 3 — The Python ML stack, takes these tools into machine learning (feature matrices — tables of numeric inputs, one row per example; scikit-learn pipelines — chained preprocessing + model steps; train/test splits — holding back some data to check the model on examples it hasn't seen). This chapter stays with general data work and does not cover scikit-learn.

Code. `chapters/18-data-stack/code/` — `numpy_basics.py`, `pandas_basics.py`, `matplotlib_demo.py`, `end_to_end.py`, and `plot_style.py` (a shared chart style). Plots are saved to `img/`. Versions used: Python 3.14.7, numpy 2.5, pandas 3.0.6, matplotlib 3.11.



Library	Core object	Think of it as
numpy	`ndarray`	A grid of numbers of one type, stored in one block of memory
pandas	`DataFrame` / `Series`	A spreadsheet table with labelled rows and columns; columns can have different types
matplotlib	`Figure` / `Axes`	A canvas with one or more plots drawn on it

18.1 numpy arrays

What it is

numpy (Numerical Python) gives you the `ndarray`: an n-dimensional array whose elements all share one **dtype** (data type such as `int64`, `float64`, `bool`) and sit side by side in one contiguous block of memory. Its **shape** is a tuple of sizes per dimension (`(3, 4)` = 3 rows, 4 columns); `ndim` is the number of dimensions.

Analogy: a Python list is a shelf of labelled boxes, each box holding any kind of object somewhere else in the warehouse; a numpy array is an egg carton — every slot the same size, all in one piece, so a machine can process a whole carton at once.

Why it matters

- **Speed**: operations run as compiled C loops over raw numbers — typically 10–100× faster than Python loops (measured below: 35×).
- **Memory**: 8 bytes per `float64` versus ~24–32 bytes per Python float object plus an 8-byte pointer.

- **Ecosystem**: pandas columns, scikit-learn (the classic ML library) inputs, images in OpenCV/Pillow (image-processing libraries), and PyTorch tensors (a deep-learning library's n-dimensional arrays, which convert to and from numpy) all build on this idea.

How it works — creating and inspecting

```
a = np.array([1, 2, 3])                       # from a list
np.zeros((2, 3)); np.ones(3, dtype=np.int32)  # filled arrays
np.arange(0, 10, 3); np.linspace(0, 1, 5)     # ranges: step-based vs count-based
m = np.arange(12).reshape(3, 4)               # 3 rows x 4 cols
np.array([1, 2.5]).dtype                      # mixed -> upcast to float64
np.array([1.7, -1.7]).astype(int)             # truncates toward zero
```

Output (`numpy_basics.py`):

```
[1 2 3] int64 (3,) 1
[[0. 0. 0.]
 [0. 0. 0.]]
[1 1 1]
[0 3 6 9]
[0.   0.25 0.5  0.75 1.  ]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
m.shape=(3, 4) m.size=12 m.dtype=int64 m.nbytes=96 itemsize=8
mixed ints and floats -> float64; astype -> [ 1 -1]
```

`reshape` needs the same total size (12 = 3 × 4); pass `-1` for one dimension to let numpy compute it (`reshape(-1, 2)`). ("Upcast" in the code above means numpy picks a wider type that can hold every value — ints and floats together become floats.) Other creators: `np.full`, `np.eye` (identity matrix: 1s on the diagonal, 0s elsewhere), `np.empty` (uninitialised — fast, contains garbage), `np.zeros_like(m)`.

Indexing, slicing, views and masks

```
m[1, 2]; m[0]; m[:, 1]; m[-1, ::2]            # element, row, column, strided
view = m[:2, :2]; view[0, 0] = 100            # slicing returns a VIEW -> m changes too
mask = m % 2 == 0; m[mask]                    # boolean mask selects matching elements
m[[2, 0]]                                     # fancy (list) indexing returns a COPY
np.where(m > 5, m, -1)                        # vectorised if/else
```

```
m[1, 2]=6  m[0]=[0 1 2 3]  m[:, 1]=[1 5 9]  m[-1, ::2]=[ 8 10]
after view[0,0]=100, m[0,0]=100  shares memory: True
even values: [ 0  2  4  6  8 10]
fancy rows [2, 0]:
[[ 8  9 10 11]
 [ 0  1  2  3]]
np.where(m > 5, m, -1):
[[-1 -1 -1 -1]
 [-1 -1  6  7]
 [ 8  9 10 11]]
```

Indexing kind	Example	Returns
Basic (ints, slices)	`m[1:, ::2]`	**View** (shares memory)
Boolean mask	`m[m > 5]`	Copy (1-D)
Fancy (int lists/arrays)	`m[[2, 0]]`	Copy

Unlike Python lists, a numpy slice is a **view** (a new array object that looks at the *same* memory as the original): changing it changes the original. ("Strided" in the code above means taking every k-th element.) Call `.copy()` when you need independence. Combine masks with `&`, `|`, `~` and parentheses: `m[(m > 2) & (m < 9)]` (`and`/`or` raise an error on arrays).

Vectorised maths and aggregation along an axis

```
x*2=[2. 4. 6. 8.]  x**2=[ 1.  4.  9. 16.]  np.sqrt(x)=[1.    1.414 1.732 2.   ]  x>2=[False False  True  True]
m.sum()=66  m.sum(axis=0)=[12 15 18 21]  m.sum(axis=1)=[ 6 22 38]
m.mean()=5.50  m.max(axis=1)=[ 3  7 11]  m.argmax()=11  cumsum=[ 1  3  6 10]
```

Arithmetic and **ufuncs** (universal functions: `np.sqrt`, `np.exp`, `np.log`, `np.abs` …) apply element-wise (to each element separately). For reductions (operations that collapse many values into one, like `sum` or `max`), `axis=0` means "collapse the rows" — one result **per column**; `axis=1` collapses the columns — one result **per row**. `argmax` gives the position of the maximum (in the flattened — treated as one long 1-D row — array unless you pass `axis`).

Vectorisation, measured

```
[v * 2 + 1 for v in big]      # Python: one bytecode loop, one int object per element
arr * 2 + 1                   # numpy: two C loops over raw int64s
```

```
1M elements: list comprehension 25.8 ms, numpy 0.75 ms, x35
```

Vectorisation means expressing a computation as whole-array operations instead of a Python loop. The rule of thumb: if you are writing `for i in range(len(arr))` over a numpy array, look for the array operation (`np.where`, masks, `cumsum`, `np.diff`, `np.clip`, broadcasting).

Other everyday array functions: **join** with `np.concatenate([a, b])` (along an existing axis) or `np.stack([a, b])` / `np.vstack` / `np.hstack` (new or row/column axis); **sort** with `np.sort(a)` (returns a copy) or `a.sort()` (in place), and `np.argsort(a)` for the order of indices (e.g. to sort one array by another); **deduplicate/count** with `np.unique(a, return_counts=True)`; **reshape views** with `a.T`, `a.ravel()`, `a[:, None]`.

Common mistakes

- Expecting a slice to be a copy (it's a view) — or the reverse with fancy indexing.
- Integer overflow (a result too big for the type's fixed number of bits): fixed-size dtypes wrap around (`np.int8(127) + np.int8(1)` → −128 — numpy warns for scalars, but array arithmetic wraps silently); Python ints never overflow.
- Comparing floats with `==` — use `np.isclose` / `np.allclose`.
- Building an array by `np.append` in a loop (copies each time, O(n²)) — collect in a list, convert once.
- Mixing up `axis=0` and `axis=1`.

TL;DR

One dtype, one memory block, shape tuple. Slices are views; masks and fancy indexing copy. Whole-array operations replace loops and run tens of times faster.

18.2 Broadcasting

What it is

Broadcasting is numpy's rule for combining arrays of different shapes without copying data: the smaller array is (virtually) stretched to match the larger one.

How it works — the rule

Compare the two shapes **from the right-most dimension leftwards**:

1. If one shape is shorter, pad it on the left with 1s.
2. Two dimensions are compatible if they are **equal** or **one of them is 1**.
3. A size-1 dimension is stretched to the other size. Any other mismatch → `ValueError`.

Worked example: `(3, 1) + (4,)` → pad → `(3, 1)` vs `(1, 4)` → result `(3, 4)`.

```
col = np.array([[10], [20], [30]])                # shape (3, 1)
row = np.array([1, 2, 3, 4])                      # shape (4,)
col + row                                         # shape (3, 4)
z = (data - data.mean(axis=0)) / data.std(axis=0)   # standardise each column: (3,2) op (2,)
np.ones((3, 2)) + np.ones(3)                      # (3,2) vs (3,) -> last dims 2 vs 3 -> error
```

```
(3,1) + (4,) -> (3, 4)
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
column-standardised:
[[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
(3,2) + (3,) -> ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

To subtract a **per-row** value, make it a column: `data - data.mean(axis=1, keepdims=True)` (or `[:, None]` / `np.newaxis` to add a dimension).

Real-world examples

Standardising features (subtract column means, divide by column stds), normalising images (`img / 255.0`, or subtracting a per-channel mean of shape `(3,)` from an `(H, W, 3)` image), pairwise distances (`a[:, None, :] - b[None, :, :]`).

Self-check

1. Shape of `(5, 1, 3) + (4, 1)`? → `(5, 4, 3)`.
2. Why does `(3, 2) + (3,)` fail? → Right-most dims 2 and 3 differ and neither is 1.
3. How do you subtract each row's mean? → `x - x.mean(axis=1, keepdims=True)`.

TL;DR

Align shapes from the right; equal or 1 is fine; size 1 stretches. `keepdims=True` or `[:, None]` fixes most shape errors.

18.3 Linear algebra and random numbers

Linear algebra basics

```
A = np.array([[3.0, 1.0], [1.0, 2.0]]); b = np.array([9.0, 8.0])
A @ A                       # matrix multiplication (same as np.matmul); A * A is element-wise!
np.linalg.solve(A, b)       # solve A x = b
np.linalg.det(A); np.linalg.inv(A); np.linalg.eigh(A); np.linalg.norm(b); A.T
```

```
A @ A =
[[10.  5.]
 [ 5.  5.]]
solve A x = b -> x = [2. 3.], check A @ x = [9. 8.]
det = 5.000, inv =
[[ 0.4 -0.2]
 [-0.2  0.6]]
eigenvalues = [1.382 3.618], norm of b = 12.042, A.T =
[[3. 1.]
 [1. 2.]]
```

(Quick glossary: the **determinant** `det` is a single number summarising a square matrix — zero means it has no inverse; the **inverse** `inv(A)` is the matrix that undoes `A`; **eigenvalues** are the factors by which `A` stretches its special directions, the eigenvectors; the **norm** is a vector's length.) Check by hand: 3·2 + 1·3 = 9 and 1·2 + 2·3 = 8. ✓ det = 3·2 − 1·1 = 5. ✓ The norm is √(9² + 8²) = √145 ≈ 12.042. ✓

Need	Use	Note
Matrix product	`A @ B`	`*` is element-wise
Solve `A x = b`	`np.linalg.solve`	Faster and more accurate than `inv(A) @ b`
Least squares	`np.linalg.lstsq`	Over-determined systems (more equations than unknowns)
Eigen-decomposition	`np.linalg.eigh` (symmetric), `eig` (general)	With numpy 2.5 here, `eig` returned a complex dtype even for this real-valued case; `eigh` returns real values sorted ascending
SVD (singular value decomposition — splitting a matrix into simpler rotation and scaling parts)	`np.linalg.svd`	Basis of PCA (principal component analysis — compressing data to its most informative directions)
Dot product / length	`a @ b`, `np.linalg.norm(a)`	

The maths behind these (vectors, matrices, eigenvalues) is covered in the ML Handbook's maths chapter.

Random numbers

```
rng = np.random.default_rng(seed=42)        # modern Generator API
rng.integers(0, 10, size=5); rng.normal(0, 1, size=3); rng.choice(["a", "b", "c"], size=4)
rng.shuffle(arr5)                           # in place
```

```
integers: [0 7 6 4 4]
normal:   [ 0.941 -1.951 -1.302]
choice:   ['c' 'c' 'c' 'c']
shuffle:  [1 0 2 4 3]
same seed, same numbers: [0 7 6 4 4]
```

Use `np.random.default_rng(seed)` (the seed is the starting value; the same seed always produces the same "random" sequence) and pass the `rng` object around. The old global functions (`np.random.seed`, `np.random.rand`) still work but share hidden global state, which makes reproducibility across modules fragile. `rng.integers(low, high)` excludes `high`. Other useful methods: `random`, `uniform`, `permutation`, `binomial`, `poisson`. (Four 'c's in a row is just chance — with this seed.)

In the interview

30-second answer: "A numpy array is a typed, contiguous block of memory with a shape, so operations run as C loops — vectorised code is often tens of times faster than Python loops. Slices are views, fancy indexing copies. Broadcasting lines up shapes from the right, and a dimension of 1 stretches. I use `@` for matrix multiply, `linalg.solve` rather than inverting, and `default_rng(seed)` for reproducible randomness."

- *Why is numpy fast?* — Contiguous typed memory, compiled loops, no per-element Python objects, SIMD (CPU instructions that process several numbers at once), and BLAS/LAPACK (decades-old, heavily optimised libraries of linear-algebra routines) for linear algebra.
- *`a * b` vs `a @ b`?* — Element-wise vs matrix product.
- *How do you make results reproducible?* — Seeded `Generator` passed explicitly.

TL;DR

`@` for matrices, `solve` not `inv`, `eigh` for symmetric matrices; `default_rng(seed)` for randomness.

18.4 pandas: Series and DataFrame

What it is

- A **Series** is a one-dimensional labelled array: values plus an **index** (the row labels).
- A **DataFrame** is a table: an ordered collection of Series (columns) sharing one index. Each column has its own dtype.

Analogy: a DataFrame is a spreadsheet in which each column is typed and every row has a name tag.

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="score")
s["b"]; s.iloc[0]; s[s > 15]; s.mean()
```

```
a    10
b    20
c    30
Name: score, dtype: int64
s['b']=20  s.iloc[0]=10  s[s > 15].tolist()=[20, 30]  s.mean()=20.0
```

Operations between Series **align on the index**: `s1 + s2` adds values with matching labels; labels present in only one become `NaN` ("not a number" — the marker for a missing value, explained below). This automatic alignment is pandas' biggest difference from numpy.

Reading data and first look

```
df = pd.read_csv(io.StringIO(CSV), parse_dates=["date"])   # usually pd.read_csv("orders.csv", ...)
df.head(3); df.dtypes; df.shape; df.describe()
```

```
   order_id       date customer   city   product   qty  price
0         1 2026-01-03      ana   Pune       pen  10.0    1.5
1         2 2026-01-03      ben  Delhi  notebook   2.0    4.0
2         3 2026-01-10      ana   Pune  notebook   NaN    4.0
order_id             int64
date        datetime64[us]
customer               str
city                   str
product                str
qty                float64
price              float64
shape=(8, 7)
       order_id                 date        qty      price
count       8.0                    8   7.000000   7.000000
mean        4.5  2026-02-03 00:00:00   6.142857   5.928571
min         1.0  2026-01-03 00:00:00   1.000000   1.500000
max         8.0  2026-03-05 00:00:00  20.000000  25.000000
```

What to notice:

- `qty` became `float64` because it has a missing value: `NaN` (Not a Number, numpy's missing marker) is a float. Use the nullable dtype (a type that can also hold "missing") `"Int64"` (capital I) to keep integers with missing values (shown as `<NA>`).
- Text columns show dtype `str` — new in pandas 3.0. Earlier versions used the generic `object` dtype. The `str` dtype holds only strings (and missing values); it is backed by PyArrow (the Python library for Apache Arrow, a fast, standard in-memory columnar data format) when installed, otherwise by Python objects.
- Dates parsed from strings default to **microsecond** resolution (`datetime64[us]`) in pandas 3.0 (earlier: nanoseconds).
- First-look checklist: `head()`, `info()` (dtypes, non-null counts, memory), `describe()` (count/mean/std — standard deviation, a measure of spread — /min/quartiles — the values 25 %, 50 % and 75 % of the way through the sorted data — /max), `value_counts()` on categorical columns (columns holding a small set of labels, like city names), `isna().sum()`.

Input and output

Format	Read	Write	Notes
CSV	`pd.read_csv(path, usecols=, dtype=, parse_dates=, chunksize=)`	`df.to_csv(path, index=False)`	Universal, slow, loses dtypes
Excel	`pd.read_excel`	`df.to_excel`	Needs `openpyxl`
JSON	`pd.read_json(lines=True)`	`df.to_json(orient="records")`	APIs, logs
Parquet	`pd.read_parquet`	`df.to_parquet`	Columnar (stores each column together, so reading a few columns is cheap), compressed, keeps dtypes, fast; needs `pyarrow`. Preferred for anything big.
SQL	`pd.read_sql(query, conn)`	`df.to_sql(name, conn)`	Via SQLAlchemy or `sqlite3`
Clipboard / HTML	`read_clipboard`, `read_html`		Quick grabs

`chunksize=100_000` in `read_csv` returns an iterator of DataFrames — process files larger than memory piece by piece.

TL;DR

Series = labelled 1-D; DataFrame = table of Series. Operations align on labels. pandas 3: text columns are `str`, dates default to microseconds. Parquet for storage, CSV for exchange.

18.5 Selecting data: `loc`, `iloc` and filters

What it is

Accessor	Selects by	Slice end	Example
`df["col"]` / `df[["a", "b"]]`	Column name(s)	—	one Series / a DataFrame
`df.loc[rows, cols]`	**Labels** (index values, column names) or boolean masks	**Inclusive**	`df.loc[1:2, ["customer", "qty"]]` → rows labelled 1 **and** 2
`df.iloc[rows, cols]`	**Integer positions**	Exclusive (like Python)	`df.iloc[0, 2]` → first row, third column
`df.at[r, c]` / `df.iat[i, j]`	One cell, fast	—	

```
df.loc[0, "customer"]; df.iloc[0, 2]; df.loc[1:2, ["customer", "qty"]]
df[(df["city"] == "Pune") & (df["product"] == "pen")]       # boolean mask, parentheses required
df.query("qty >= 5 and price < 2")                          # same idea as a string expression
df[df["product"].isin(["bag", "notebook"])]
```

```
ana | ana | {'customer': ['ben', 'ana'], 'qty': [2.0, nan]}
{'order_id': [1, 8], 'qty': [10.0, 20.0]}
[1, 4, 8]
[2, 3, 5, 6, 7]
```

Other handy filters: `.between(a, b)`, `.str.contains("x")`, `.str.startswith`, `.isna()`/`.notna()`, `~mask` for NOT.

Sorting and ranking: `df.sort_values("revenue", ascending=False)` (or a list of columns), `df.sort_index()`, `df.nlargest(3, "revenue")` for a quick top-N, and `df["city"].value_counts()` (frequency of each value, most common first; `normalize=True` for shares).

Setting values and Copy-on-Write

Always set with **one** `.loc` call:

```
df.loc[df["revenue"] >= 20, "tier"] = "big"      # correct
# df["tier"][df["revenue"] >= 20] = "big"       # chained assignment: never modifies df in pandas 3
```

In **pandas 3.0, Copy-on-Write (CoW)** is always on: any result of indexing *behaves as a copy*. Changing it never changes the original DataFrame, and "chained assignment" (`df[a][b] = v`) does nothing to `df` (pandas warns). The old `SettingWithCopyWarning` is gone. Internally pandas still shares memory until one side is modified, so it stays fast.

```
sub = df[df["city"] == "Pune"]
sub["qty"] = 0                                  # modifies only `sub`, never df
```

```
df Pune qty still: [10, 0, 2, 20]
```

(The 0 is the order whose missing quantity was filled with 0 earlier.)

Common mistakes

- Using `and`/`or` instead of `&`/`|` → `ValueError: The truth value of a Series is ambiguous`.
- Forgetting parentheses around each condition (`&` binds tighter than `==`).
- Assuming `loc` slices exclude the end label.
- Relying on chained assignment (silently ineffective in pandas 3).

TL;DR

`loc` = labels (inclusive), `iloc` = positions (exclusive). Filter with masks joined by `&`/`|`. Assign with a single `.loc[mask, col] = value`.

18.6 Cleaning: missing data and new columns

Missing data

pandas marks missing values as `NaN` (floats), `NaT` (datetimes) or `pd.NA` (nullable dtypes); `None` in input becomes one of these.

Task	Method
Find	`df.isna()`, `df.isna().sum()`, `df.info()`
Drop	`df.dropna()` (any NaN in row), `dropna(subset=["qty"])`, `dropna(thresh=5)`
Fill with constant	`df["qty"].fillna(0)`
Fill with statistic	`fillna(df["price"].median())` or per group with `transform` (below)
Fill from neighbours	`ffill()` (forward-fill: copy the previous value down), `bfill()` (backward-fill), `interpolate()` (estimate in-between values; time series — data recorded over time)
Replace sentinel values (placeholder codes such as -1 meaning "unknown")	`df.replace(-1, np.nan)`

```
df["qty"] = df["qty"].fillna(0).astype(int)
df["price"] = df["price"].fillna(df.groupby("product")["price"].transform("median"))
```

```
{'qty': 1, 'price': 1}
after fill: nulls=0, order 6 (missing price, bag) filled with bag median=25.0
```

The missing bag price was filled with the median (middle value when sorted) price of *bags*, not of all products — group-aware filling is usually more sensible. Choosing between dropping and filling is a modelling decision: drop when few rows are affected at random; fill (and possibly add an "was_missing" flag column) when missingness is common or informative. Aggregations skip `NaN` by default (`sum`, `mean`), and `NaN != NaN`, so always test with `isna()`.

New columns, vectorised

```
df = df.assign(revenue=df["qty"] * df["price"], month=df["date"].dt.to_period("M"))
df.loc[df["revenue"] >= 20, "tier"] = "big"
df["tier"] = df["tier"].fillna("small")
```

```
   order_id  revenue    month   tier
0         1     15.0  2026-01  small
1         2      8.0  2026-01  small
2         3      0.0  2026-01  small
3         4      7.5  2026-02  small
```

`assign` returns a new DataFrame, so it chains nicely. For several conditions use `np.select([cond1, cond2], [val1, val2], default=...)`; for bins (ranges that group numbers, e.g. 0–10, 10–20) use `pd.cut` (fixed edges) or `pd.qcut` (quantiles — edges chosen so each bin holds the same number of rows). Other cleaning tools: `astype`, `pd.to_numeric(errors="coerce")` (bad strings → NaN), `.str.strip().str.lower()`, `rename(columns=...)`, `drop_duplicates()`, `drop(columns=...)`.

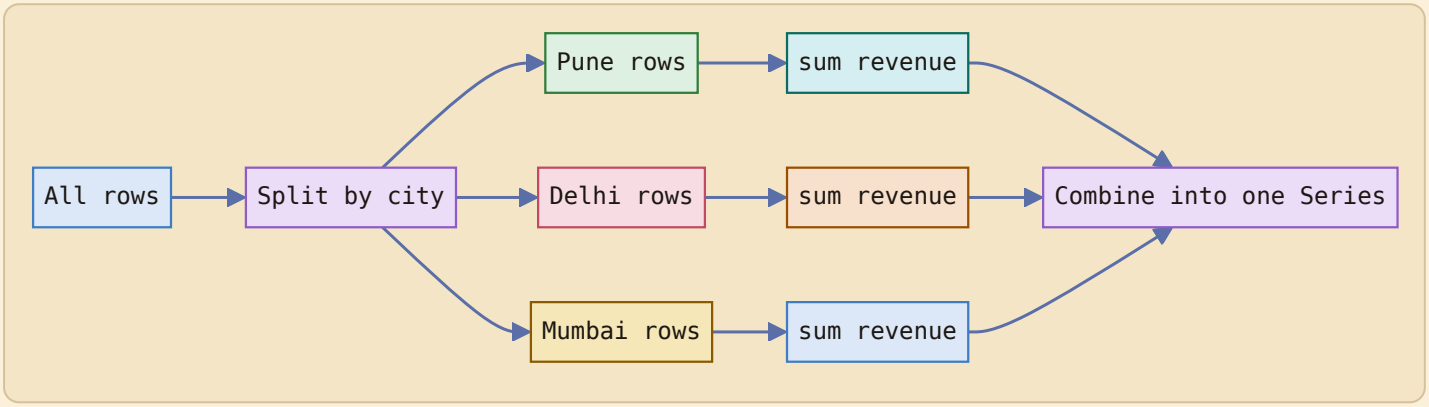
TL;DR

Find with `isna().sum()`, then drop or fill thoughtfully (group medians, forward-fill for time series). Build columns with vectorised expressions and `.loc`; never loop row by row.

18.7 `groupby`: split–apply–combine

What it is

`groupby` implements **split–apply–combine**: split rows into groups by key(s), apply a function to each group, combine the results. It's SQL's `GROUP BY` (SQL is the standard query language for databases).



How it works

```
df.groupby("city")["revenue"].sum().sort_values(ascending=False)
df.groupby("customer").agg(orders=("order_id", "count"),          # named aggregation
                           revenue=("revenue", "sum"),
                           avg_qty=("qty", "mean"))
df["share_of_city"] = df["revenue"] / df.groupby("city")["revenue"].transform("sum")
```

```
city
Pune      95.0
Delhi     33.0
Mumbai    19.5
Name: revenue, dtype: float64
          orders  revenue  avg_qty
customer
ana            3     45.0     10.0
ben            2     33.0      1.5
cara           2     19.5      4.0
dev            1     50.0      2.0
   order_id   city  share_of_city
0         1   Pune           0.16
1         2  Delhi           0.24
2         3   Pune           0.00
```

Method	Output shape	Use
`agg` / `sum` / `mean` / `size` / `count`	One row **per group**	Summaries
`transform`	Same length as the input (group result broadcast back to each row)	Group-relative features: share of total, deviation from group mean, group-wise fill
`filter`	Subset of original rows	Keep groups meeting a condition (`lambda g: len(g) > 1`)
`apply`	Anything	Flexible but slowest — last resort

`size()` counts rows including NaN; `count()` counts non-null values. Group by several keys with a list (`["city", "product"]`) → a MultiIndex (an index with several levels of labels) result; `.reset_index()` flattens it, or pass `as_index=False`. With categorical keys, pass `observed=True` to skip category combinations that never occur.

Real-world examples

Revenue per region per month, average session length per user cohort (a group of users who share a starting point, e.g. signed up in the same month), top product per store, feature engineering (creating useful input columns for a model) such as "user's average order value" for ML models.

TL;DR

`groupby(keys)[col].agg(...)` for summaries; `transform` to broadcast group stats back to rows; avoid `apply` when a built-in aggregation exists.

18.8 Combining tables: `merge`, `join`, `concat`

What it is

- `merge` joins two tables on key columns (columns whose values match rows across tables, like a customer ID), like SQL `JOIN`.
- `join` is a shortcut for merging on the index.
- `concat` stacks tables vertically (more rows) or side by side (`axis=1`).

```
customers = pd.DataFrame({"customer": ["ana", "ben", "cara", "eve"],
                          "segment": ["retail", "business", "retail", "business"]})
df.merge(customers, on="customer", how="inner")      # only matching keys
df.merge(customers, on="customer", how="left")       # keep all orders
customers.merge(df[["customer"]].drop_duplicates(), on="customer", how="outer", indicator=True)
pd.concat([df, april], ignore_index=True)            # stack rows; missing columns -> NaN
```

```
inner rows=7, left rows=8, dev's segment=nan
  customer   segment      _merge
0      ana    retail        both
1      ben  business        both
2     cara    retail        both
3      dev       NaN  right_only
4      eve  business   left_only
concat rows: 9
```

how=	Keeps	SQL
`inner` (default)	Keys in both	`INNER JOIN`
`left`	All left rows	`LEFT JOIN`
`right`	All right rows	`RIGHT JOIN`
`outer`	All keys from both	`FULL OUTER JOIN`
`cross`	Every pair	`CROSS JOIN`

`indicator=True` adds a `_merge` column that tells you where each row came from — a quick data-quality check ("which orders have no customer record?"). Use `validate="many_to_one"` to make pandas raise if the right table unexpectedly has duplicate keys, which would otherwise **multiply** rows silently. Different key names: `left_on=` / `right_on=`. Overlapping column names get `suffixes=("_x", "_y")`.

Common mistakes

- Row explosion from duplicate keys (check `len` before and after, or `validate=`).
- Keys with different dtypes (`int` vs `str`) → no matches.
- `concat` in a loop (quadratic) — collect DataFrames in a list and concat once.

TL;DR

`merge` = SQL join (choose `how`, check with `indicator`/`validate`); `concat` = stack.

18.9 Reshaping: `pivot_table` and `melt`

What it is

- **Wide** format: one row per entity, one column per variable (a spreadsheet report).
- **Long** ("tidy") format: one row per observation (one measured fact) — `(city, product, revenue)`. Plotting and grouping libraries usually prefer long.
- `pivot_table` goes long → wide (aggregating duplicates); `melt` goes wide → long.

```
pivot = df.pivot_table(index="city", columns="product", values="revenue",
                       aggfunc="sum", fill_value=0)
long = pivot.reset_index().melt(id_vars="city", var_name="product", value_name="revenue")
```

```
product   bag  notebook   pen
city
Delhi    25.0       8.0   0.0
Mumbai    0.0      12.0   7.5
Pune     50.0       0.0  45.0
     city product  revenue
0   Delhi     bag     25.0
1  Mumbai     bag      0.0
2    Pune     bag     50.0
```

Related: `pivot` (no aggregation — fails on duplicate pairs), `stack`/`unstack` (move a level between index and columns), `pd.crosstab` (frequency tables), `margins=True` in `pivot_table` for totals.

TL;DR

`pivot_table` = Excel pivot (long → wide with aggregation); `melt` = unpivot (wide → long).

18.10 Dates and times

How it works

- Parse: `pd.to_datetime(col, format="%Y-%m-%d", errors="coerce")` or `parse_dates=` in `read_csv`. Giving `format` is faster and avoids day/month ambiguity.
- The `.dt` accessor: `.dt.year`, `.dt.month`, `.dt.day_name()`, `.dt.hour`, `.dt.to_period("M")`, `.dt.floor("h")`.
- Arithmetic: `date + pd.Timedelta(days=7)`, `df["end"] - df["start"]` → timedelta, `pd.date_range("2026-01-01", periods=3, freq="D")`.
- Time series: set a `DatetimeIndex`, then `resample("ME")` (month end), `"W"`, `"D"`, `"h"` to aggregate by period, and `rolling(7).mean()` for moving averages.
- Time zones: `tz_localize("Asia/Kolkata")` then `tz_convert("UTC")`; pandas 3 uses the standard-library `zoneinfo`.

```
df["date"].dt.day_name().head(5); df["date"].dt.month.unique()
df.set_index("date")["revenue"].resample("ME").sum()
```

```
['Saturday', 'Saturday', 'Saturday', 'Sunday', 'Saturday'] [1, 2, 3]
date
2026-01-31    23.0
2026-02-28    82.5
2026-03-31    42.0
Freq: ME, Name: revenue, dtype: float64
```

Version note: pandas 3.0 removed the old frequency aliases — month end is `"ME"` (not `"M"`), quarter end `"QE"`, year end `"YE"`, hour `"h"`. Old tutorials using `resample("M")` now fail.

TL;DR

Parse early with an explicit format, use `.dt` for parts, `resample` for calendar buckets, `rolling` for moving windows. Use `"ME"`/`"h"` aliases in pandas 3.

18.11 `apply` vs vectorised, and performance tips

Measured

```
big.apply(lambda r: r["qty"] * r["price"], axis=1)   # a Python function call per row
big["qty"] * big["price"]                            # one C loop per column
```

```
100k rows: apply(axis=1) 210 ms, vectorised 0.08 ms, x2523
```

`apply(axis=1)` builds a Series object for every row and calls your Python function on it — it's a loop with extra overhead. Row iteration with `iterrows()` is even slower; if you truly must loop, `itertuples()` is much faster.

Performance checklist

Tip	Why
Vectorise: arithmetic on columns, `np.where` / `np.select`, `.str` and `.dt` methods	C loops instead of Python calls
Right dtypes: `category` for low-cardinality (few distinct values) text, smaller numeric types (`float32`, `int32`) where precision allows	Less memory, faster group-bys
Read less: `usecols=`, `dtype=`, `nrows=`, `chunksize=`	Don't load what you don't need
Store as Parquet	Faster reads, keeps dtypes, column pruning (skipping columns you don't ask for)
Use built-in aggregations (`"sum"`, `"mean"`) rather than `lambda` in `agg`	Built-ins run in optimised (Cython) code
Avoid growing DataFrames in a loop	Collect rows/frames in a list, build once
`merge` on the right keys, with `validate`	Avoid accidental row explosions
Check `df.memory_usage(deep=True)`	Find heavy columns
When data outgrows pandas	Polars (a fast DataFrame library written in Rust), DuckDB (an embedded analytics database), Dask or Spark (spread work over many cores or machines)

```
cities = pd.Series(rng.choice(["Pune", "Delhi", "Mumbai"], 100_000))
cities.memory_usage(deep=True); cities.astype("category").memory_usage(deep=True)
```

```
dtype str: 5.40 MB, category: 0.10 MB
```

A **categorical** stores each distinct value once plus a small integer code per row — here 54× less memory. (With PyArrow installed, the `str` dtype itself is also far more compact than Python string objects.)

In the interview

30-second answer: "pandas is fast when you stay vectorised: column arithmetic, `np.where`, `.str` / `.dt` accessors and built-in group aggregations all run in compiled code. `apply(axis=1)` and `iterrows` are Python loops. For memory I use categoricals and smaller dtypes, read only needed columns and store Parquet. In pandas 3, Copy-on-Write means every indexing result behaves as a copy, so I always assign with a single `.loc`."

- *`loc` vs `iloc`?* — Labels (inclusive end) vs positions (exclusive end).
- *`transform` vs `agg`?* — `transform` returns one value per original row; `agg` one per group.
- *How do you find orders with no matching customer?* — Left merge with `indicator=True` and filter `left_only`.

Self-check

1. Why does `qty` load as float? → It contains `NaN`, which is a float.
2. What does `how="left"` keep? → Every row of the left table.
3. What changed about chained assignment in pandas 3? → It never updates the original (Copy-on-Write).
4. Month-end frequency alias in pandas 3? → `"ME"`.

5. Faster way to compute `qty * price` than `apply`? → `df["qty"] * df["price"]`.

TL;DR

Vectorise, pick dtypes, read less, store Parquet, don't loop. `apply` is the last resort.

18.12 matplotlib: Figure, Axes and the four workhorse charts

What it is

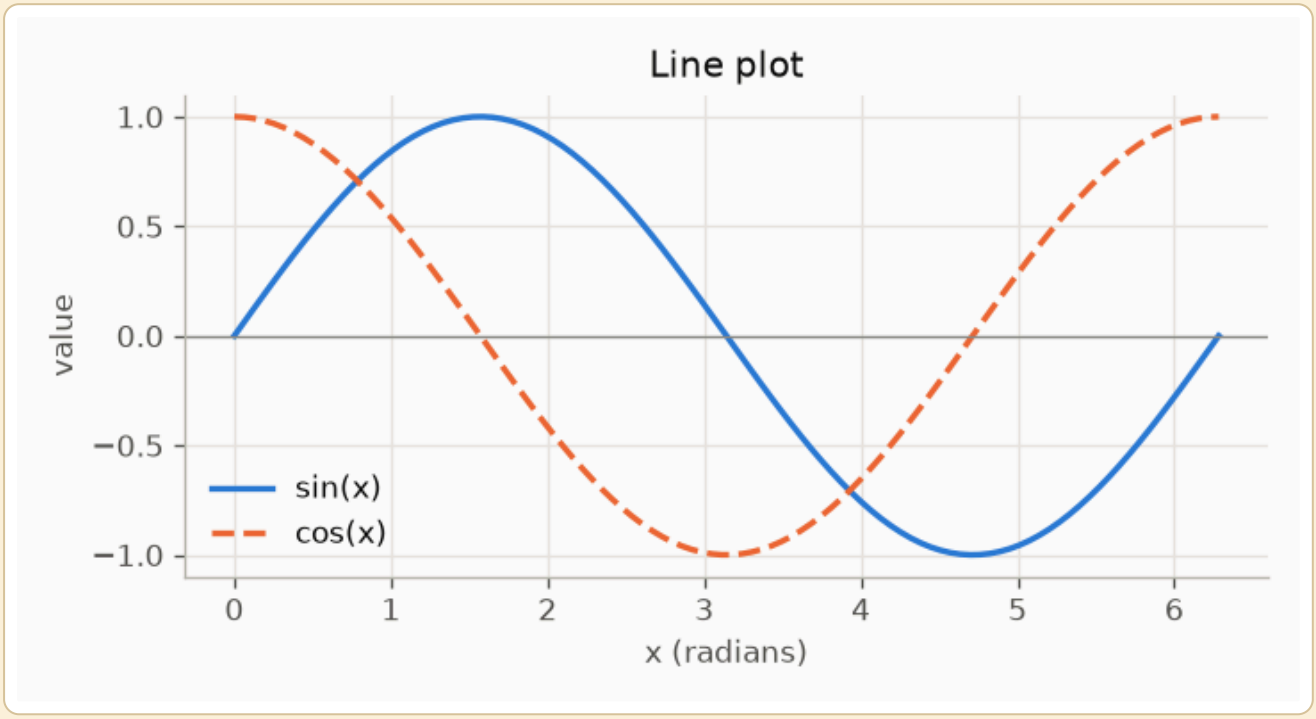
matplotlib is Python's foundational plotting library (pandas' `.plot()` and seaborn — a higher-level statistical charting library — are built on it).

- A **Figure** is the whole canvas/image.
- An **Axes** is one plot inside it — with its own x/y axis, title, labels and legend. A figure can hold many axes.
- Two styles: the **pyplot state machine** (it silently remembers a "current" figure and axes between calls: `plt.plot(...)` acts on "the current axes" — quick in notebooks) and the **object-oriented (OO) API** (you hold the figure and axes objects and call methods on them: `fig, ax = plt.subplots(); ax.plot(...)` — explicit, recommended for scripts and multiple plots). Use the OO API.

How it works

```
import matplotlib
matplotlib.use("Agg")                          # render to files only (servers, scripts)
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 3.2))       # Figure = canvas, Axes = one plot
ax.plot(x, np.sin(x), label="sin(x)")
ax.plot(x, np.cos(x), label="cos(x)", linestyle="--")
ax.set(title="Line plot", xlabel="x (radians)", ylabel="value")
ax.legend(frameon=False)
fig.tight_layout()                             # stop labels overlapping
fig.savefig(IMG / "line_plot.png", dpi=120)    # format from the extension: png, svg, pdf
plt.close(fig)                                 # free memory in scripts/loops
```



```
fig, axes = plt.subplots(2, 2, figsize=(9, 6))              # axes is a 2x2 numpy array
axes[0, 0].plot(months, sales, marker="o")                  # line: change over time
axes[0, 1].bar(["pen", "notebook", "bag"], [95, 20, 75])    # bar: compare categories
axes[1, 0].scatter(h, w, s=14, alpha=0.6)                   # scatter: relationship
axes[1, 1].hist(rng.normal(50, 10, 1_000), bins=30)         # histogram: distribution
fig.suptitle("Four basic chart types")
```

Output (`matplotlib_demo.py`):

```
corr(height, weight) = 0.80
saved line_plot.png, four_charts.png
```

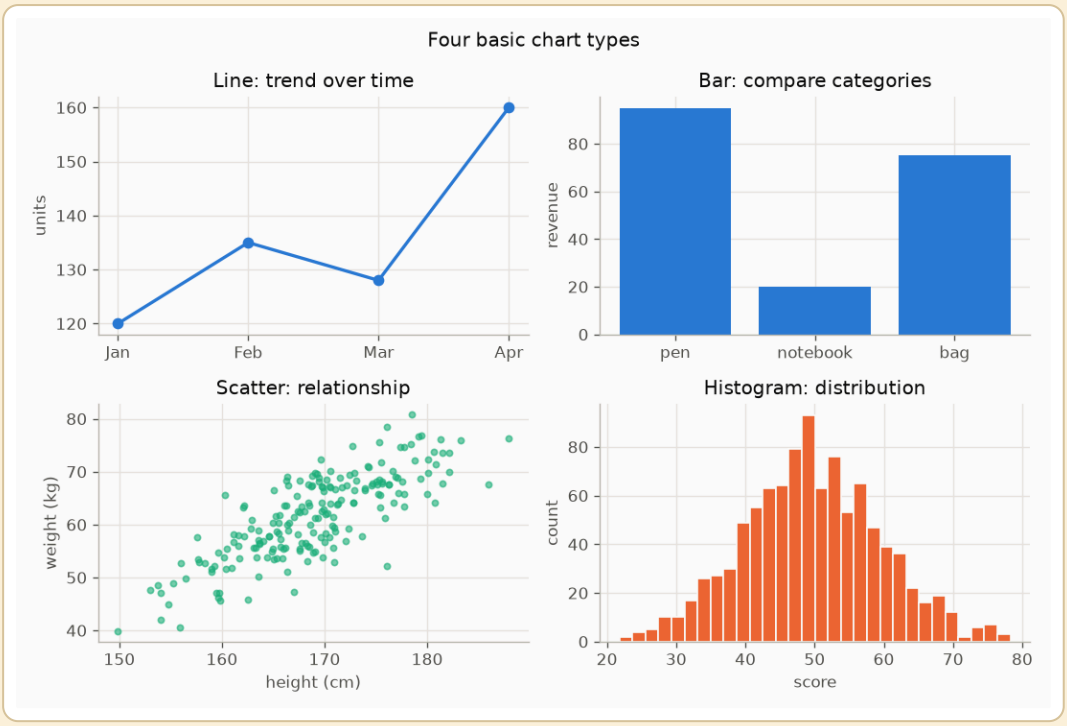


Chart	Answers	Call	Tip
Line	How does it change over an ordered x (time)?	`ax.plot(x, y)`	Don't use for unordered categories
Bar	How do categories compare?	`ax.bar` / `ax.barh`	Start the y-axis at 0; horizontal bars for long labels
Scatter	Are two numbers related?	`ax.scatter(x, y)`	`alpha` (transparency) for overplotting (so many points on top of each other that you can't see density)
Histogram	How are values distributed?	`ax.hist(values, bins=30)`	Try several `bins` values
Box plot	Spread and outliers (values far from the rest) by group	`ax.boxplot`	

Labelling essentials: `ax.set_title`, `ax.set_xlabel`/`ax.set_ylabel` (include units), `ax.legend()` whenever there are two or more series, `ax.tick_params(axis="x", rotation=30)` for crowded dates, `ax.set_xlim`/`set_ylim`, `ax.axhline` for reference lines. `savefig(..., bbox_inches="tight")` trims margins; `dpi=150–300` (dots per inch — image resolution) for print. Shortcut from pandas: `df.plot(kind="bar", ax=ax)`.

Common mistakes

- Mixing pyplot and OO calls, so labels land on the wrong subplot.
- Forgetting `plt.close(fig)` in loops → memory grows and matplotlib warns about too many open figures.

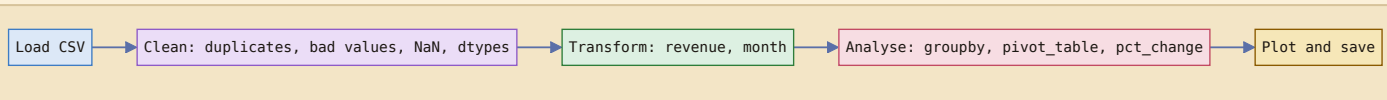
- Calling `plt.show()` in a headless script (one running with no screen, e.g. on a server) instead of `savefig` (use the `Agg` backend — the renderer that draws straight to image files).
- Two y-axes (`twinx`) for unrelated measures — usually misleading; use two panels.
- Rainbow colour maps (the mapping from numbers to colours) for ordered data — use a single-hue sequential map (e.g. `viridis`).

TL;DR

`fig, ax = plt.subplots()`, draw on `ax`, label everything, `fig.savefig(path, dpi=...)`, `plt.close(fig)`. Line = trend, bar = compare, scatter = relationship, histogram = distribution.

18.13 Mini end-to-end analysis

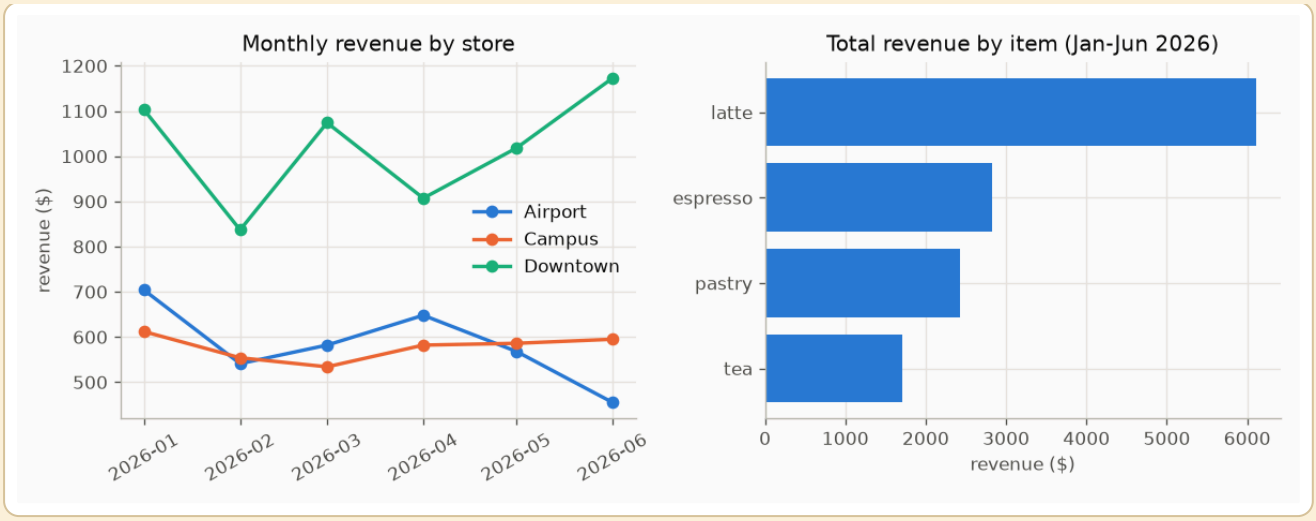
A coffee chain with three stores wants to know: which store earns most, how revenue moved month by month, and which item matters most. The script generates a realistic messy CSV (6 months, 2,000 orders, 3 % missing quantities, a negative price, 10 duplicated rows) and then runs the standard pipeline.




```
# 1. Load
raw = pd.read_csv(io.StringIO(make_raw_csv()), parse_dates=["date"])
# 2. Clean
df = (raw.drop_duplicates()
         .query("unit_price > 0")
         .assign(qty=lambda d: d["qty"].fillna(d["qty"].median()).astype(int),
                 store=lambda d: d["store"].astype("category"),
                 item=lambda d: d["item"].astype("category")))
# 3. Transform
df["revenue"] = df["qty"] * df["unit_price"]
df["month"] = df["date"].dt.to_period("M")
# 4. Analyse
by_store = df.groupby("store", observed=True).agg(orders=("qty", "size"),
                                                  revenue=("revenue", "sum"),
                                                  avg_ticket=("revenue", "mean")).round(2)
monthly = df.pivot_table(index="month", columns="store", values="revenue",
                         aggfunc="sum", observed=True).round(0)
growth = monthly.sum(axis=1).pct_change().mul(100).round(1)
# 5. Plot  (line per store + horizontal bar per item, saved to img/coffee_analysis.png)
```

Output (`end_to_end.py`):

```
raw: 2010 rows, missing qty=65, duplicates=10, negative prices=2
clean: 1999 rows, nulls=0
          orders  revenue  avg_ticket
store
Downtown    1010   6112.0        6.05
Airport      412   3497.2        8.49
Campus       577   3461.5        6.00
store    Airport  Campus  Downtown
month
2026-01    704.0   612.0    1103.0
2026-02    541.0   554.0     837.0
2026-03    582.0   534.0    1074.0
2026-04    648.0   582.0     907.0
2026-05    568.0   586.0    1018.0
2026-06    455.0   595.0    1173.0
best item per store: {'Airport': 'latte', 'Campus': 'latte', 'Downtown': 'latte'}
month-over-month total revenue change %: {'2026-02': -20.1, '2026-03': 13.4, '2026-04': -2.4, '2026-05': 1.6,
'2026-06': 2.3}
saved coffee_analysis.png
```



Reading the results:

1. **Data quality** — 10 duplicates removed (the duplicated block contained the bad row, hence 2 negative prices, 1 after dedup); 1 negative price dropped; 65 missing quantities filled with the median. Always report what cleaning did.
2. **Stores** — Downtown has the most orders and revenue; Airport has the fewest orders but the highest average ticket (8.49 vs ~6), which matches the 40 % airport markup (higher price) built into the data. (`pct_change` in the code computes the percentage change from one row to the next.)
3. **Trend** — February dropped 20 % (it has 28 days, and our synthetic data is uniform per day — a reminder to normalise by days or compare like periods); Airport declines towards June while Downtown grows.
4. **Items** — lattes lead in every store: highest price among drinks and most frequent item.

This is the workflow of most analytics tasks and the first half of any ML project; the ML Handbook's Chapter 3 continues from a clean DataFrame to features, models and evaluation.

18.14 Chapter self-check

1. Why is `arr * 2` faster than a list comprehension? → One compiled loop over contiguous typed memory, no per-element Python objects.
2. Is `m[:2, :2]` a copy? → No, a view.
3. Result shape of `(4, 1) * (3,)`? → `(4, 3)`.
4. Better than `inv(A) @ b`? → `np.linalg.solve(A, b)`.
5. How do you get reproducible random numbers? → `rng = np.random.default_rng(seed)`.
6. `df.loc[1:3]` vs `df.iloc[1:3]` with a default index? → Labels 1–3 (3 rows) vs positions 1–2 (2 rows).
7. How do you add each row's share of its group total? → `col / df.groupby(key)[col].transform("sum")`.
8. How do you detect keys missing from a lookup table after a merge? → `indicator=True`, look for `left_only`.
9. Long → wide? Wide → long? → `pivot_table`; `melt`.
10. Figure vs Axes? → The canvas vs one plot on it.

Chapter TL;DR

- **numpy**: typed contiguous arrays; shape/dtype; slices are views; masks and fancy indexing copy; vectorise instead of looping (35× here); `axis=0` per column, `axis=1` per row; broadcasting aligns shapes from the right; `@`, `solve`, `eigh`; `default_rng(seed)`.
- **pandas**: Series/DataFrame with label alignment; `read_csv`/`read_parquet`; `loc` (labels, inclusive) vs `iloc` (positions); masks with `&`/`|`; fill missing values thoughtfully; `groupby` + `agg`/`transform`; `merge` with `how`, `indicator`, `validate`; `pivot_table`/`melt`; `.dt` and `resample("ME")`.

- **pandas 3.0**: `str` dtype for text, Copy-on-Write always on (assign with one `.loc`), microsecond datetimes, new frequency aliases.
- **Performance**: vectorise (2,500× over `apply` here), categoricals (54× less memory here), Parquet, read less.
- **matplotlib**: OO API (`fig, ax = plt.subplots()`), line/bar/scatter/hist, label everything, `savefig` + `close`.
- **Workflow**: load → clean → transform → analyse → plot → explain, then hand the clean table to ML.